Computer organisation v/s Computer Architecture :

* Computer organisation :

- Refers to the operational units and their interconnection that realize the architectural specifications.
- It focuses on the physical aspects like hardware components, data paths, control signals, memory types, etc.
- ex: The functioning and interconnecting of the control unit, ALU, registers and buses

* Computer Architecture :

- Refers to the abstract model & functional description of requirements & design implementations ~~and design~~ for various parts of a computer.
- It deals with the structure & behaviour as seen by the programmer including instruction sets, addressing modes, data types, etc
- ex: Instruction set ~~architetu~~ architecture (ISA), data types supported, addressing modes.

Program and Von-Neumann Architecture :

* Program :

- A set of instructions that a computer follows to perform a specific task.
- In memory, it consists of instructions & data stored in a sequence.

* Von-Neumann Architecture :

- A computer architecture model where data & instructions are stored in the same memory space.
- It uses a single bus for accessing both instructions & data, leading to the concept of the "stored program"

20/20

G. Yaswanth

Key components of Von-Neumann Architecture :-

(1) Memory :

- stores both program instructions and data.
- Addressed by the CPU for reading and writing data.

(2) CPU (central processing unit):

- MAR (memory address register):- Holds the address of the memory location to be accessed.
- MBR (memory buffer register):- Temporarily holds data read from & written to memory.
- I/O AR (I/O (~~in & out~~) address register) :- Holds the address of the I/O (input - output) device
- I/O BR (I/O buffer register):- Temporarily holds data transferred to (& from) an I/O (input - output) device
- PC (program counter):- Holds the address of the next instruction to be executed.
- IR (instruction register):- Holds the currently fetched instruction.
- Execution unit (ALU):- performs ~~all~~ arithmetic & logic operations.

(3) I/O module :

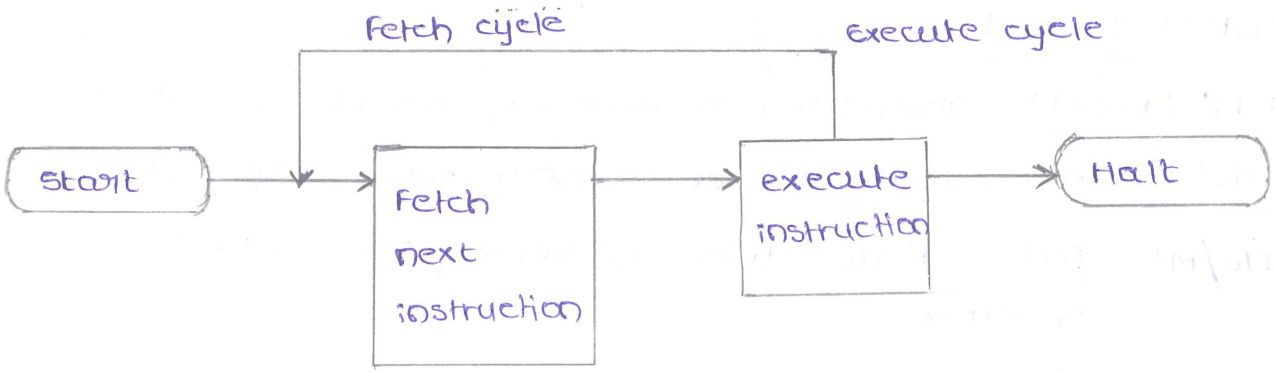
- Interfaces the CPU with peripheral devices like disks, keyboards and printers
- Transfers data between CPU & peripheral devices.

Basic Instruction Cycle :

* The instruction ~~cycle~~ cycle comprises two main phases :

(1) the Fetch cycle

(2) the Execution cycle



Basic Instruction Cycle

(1) Fetch Cycle :

- The PC holds the address of next instruction. The address is loaded into the MAR
- The control unit sends a read signal (RD'). The instruction is fetched from memory & loaded into MBR
- The fetched instruction is moved from MBR to IR. The PC is incremented to point to the next instruction.

(2) Execution Cycle :

- The instruction in the IR is decoded & the necessary operations are performed.
- Example operations are

(a) If it is an ALU operation, the ALU performs calculation
(b) If it involves memory, a memory read & write operation is performed.
(c) If it involves an I/O operation, data is transferred to/from an I/O device.

Data Transfer Mechanism :

* Processor - Memory Transfer :

- Data transfer between the CPU & main memory
- Utilizes the address bus (to specify the memory location) and data bus (to transfer the data).

* Processor - I/O Transfer :

- Data transfer between the CPU & I/O devices
- Uses I/O instructions and the data bus.

Control signals :

* RD' (read) : Indicates a memory read operation
* WR' (write) : Indicates a memory write operation
* I/O/M' : Differentiates between memory (0) & I/O (1) operations

~~In gene~~

Action categories :

* In general, these actions fall into four categories :

(1) Processor - memory Transfer
(2) Processor - I/O Transfer
} → (these two are in the data transfer mechanism)

(3) Data ~~transfer~~ processing : The processor may perform some arithmetic & logic operation on data
(4) Control : An instruction may specify that the sequence of execution be altered.

Q : What is Von - Neumann architecture ?

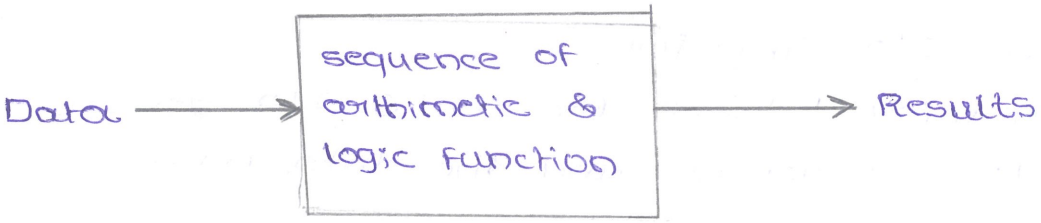
A : It consists of 3 concepts :

* Data & instructions are stored in single read - write memory
* Contents of memory are addressable by location, without regard to the type of data contained there.
* Execution occurs in a sequential fashion (unless explicitly modified) from one instruction to next.

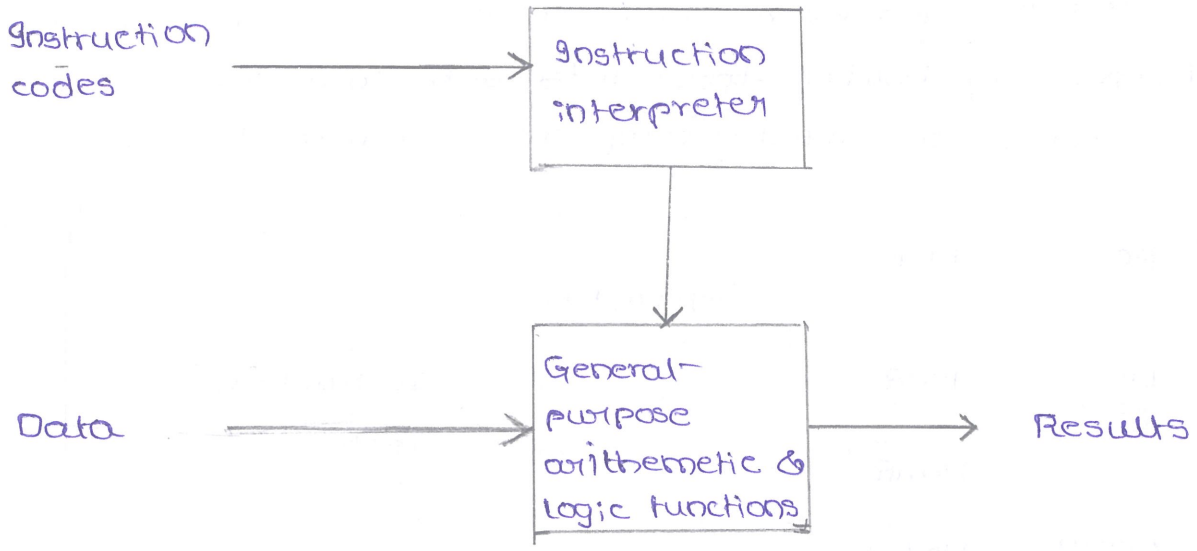
Q : What are software and hardware approaches ?

A : These two diagrams represents the software and hardware approaches :

(i) Software approach (i.e ; programming in software)



(ii) Hardware approach (i.e; programming in hardware)



Q: What are the basic components of computer system ?

A: The basic components of computer system are :

CPU - control unit (CU)

I/O - input module - keyboard
output module - monitor

Memory - MAR, MBR, I/O AR, I/O BR

Q: What are the four basic operations of computer system ?

A: The four basic operations of computer system are :

(1) Memory Address Register (MAR) : specifies address in memory for next read (or write)

(2) Memory Buffer Register (MBR) : contains data to be written into memory or recieves data read from memory.

(3) I/O Address Register (I/O AR) : specifies a particular I/O device

(4) I/O Buffer Register (I/O BR) : used for exchange of data between an I/O module & CPU

Computer components : Top-level view

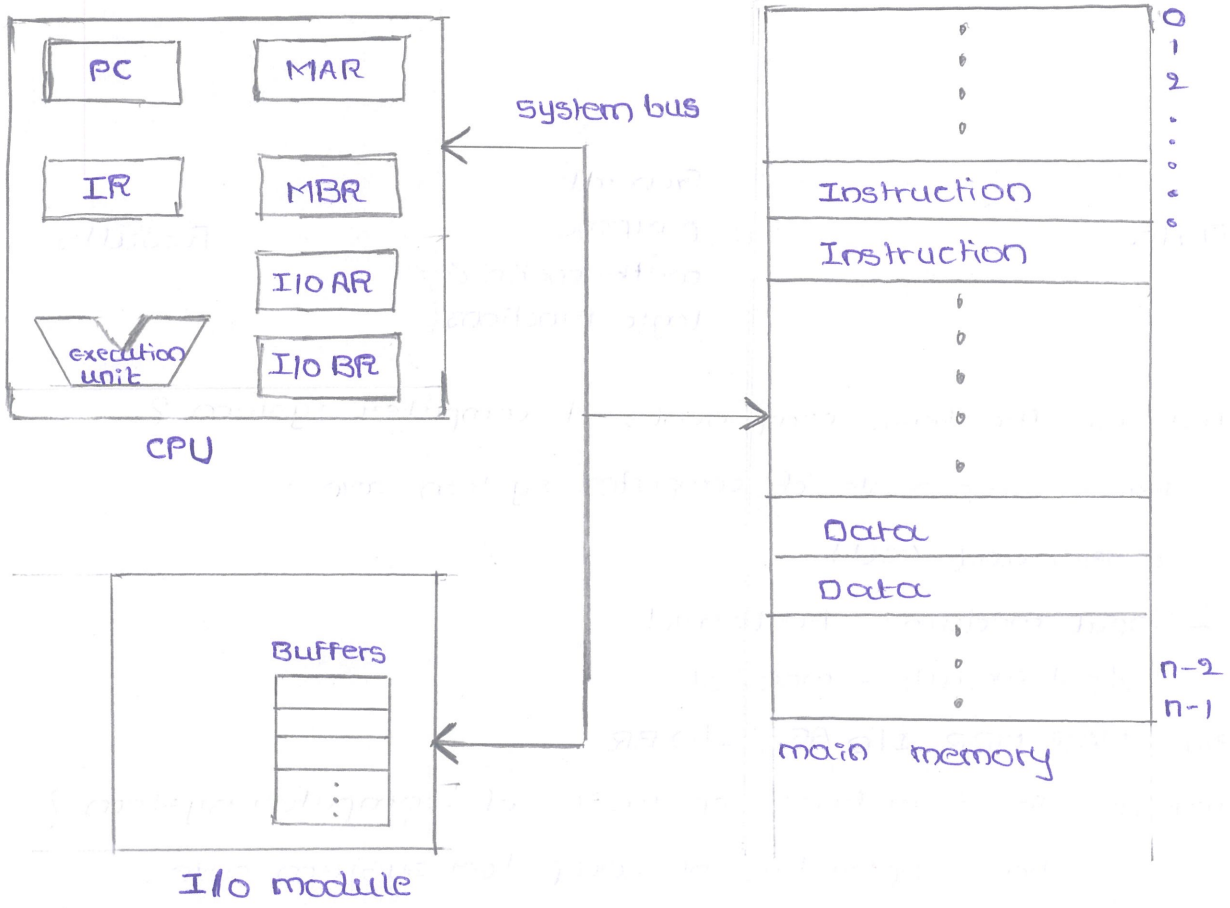
* A memory module consists of a set of locations, defined by sequentially numbered addresses.

* Each location contains a binary number that can be

interpreted as either an instruction or data.

* An I/O module transfers data from external devices to CPU & memory, and viceversa. It contains internal buffers for temporarily holding these data until they can be sent on.

* The diagram of computer components - top level view :



To understand the components & their interactions :

Ex: Fetching & executing an instruction

(1) PC (program counter) : The PC holds the address of the next instruction to be fetched from memory. Initially, it might point to the first instruction of program.

(2) MAR : The value of PC is transferred to MAR to specify the memory location where instruction is stored.

(3) System Bus : The value of MAR is sent through the system bus to the main memory

(4) Main memory : The memory fetches the instruction at the address specified by MAR & places it on the data bus.

(5) MBR : The fetched instruction is temporarily stored in MBR

(6) IR : The instruction from MBR is transferred to IR for decoding

(7) Instruction decoding : The CPU decodes the instruction to determine operation to be performed & address of any operands.

(8) Operand Fetch : If instruction requires operands, the CPU fetches from memory using same process as fetching the instruction

(9) Execution unit : The CPU's execution unit performs the specified operation on operands

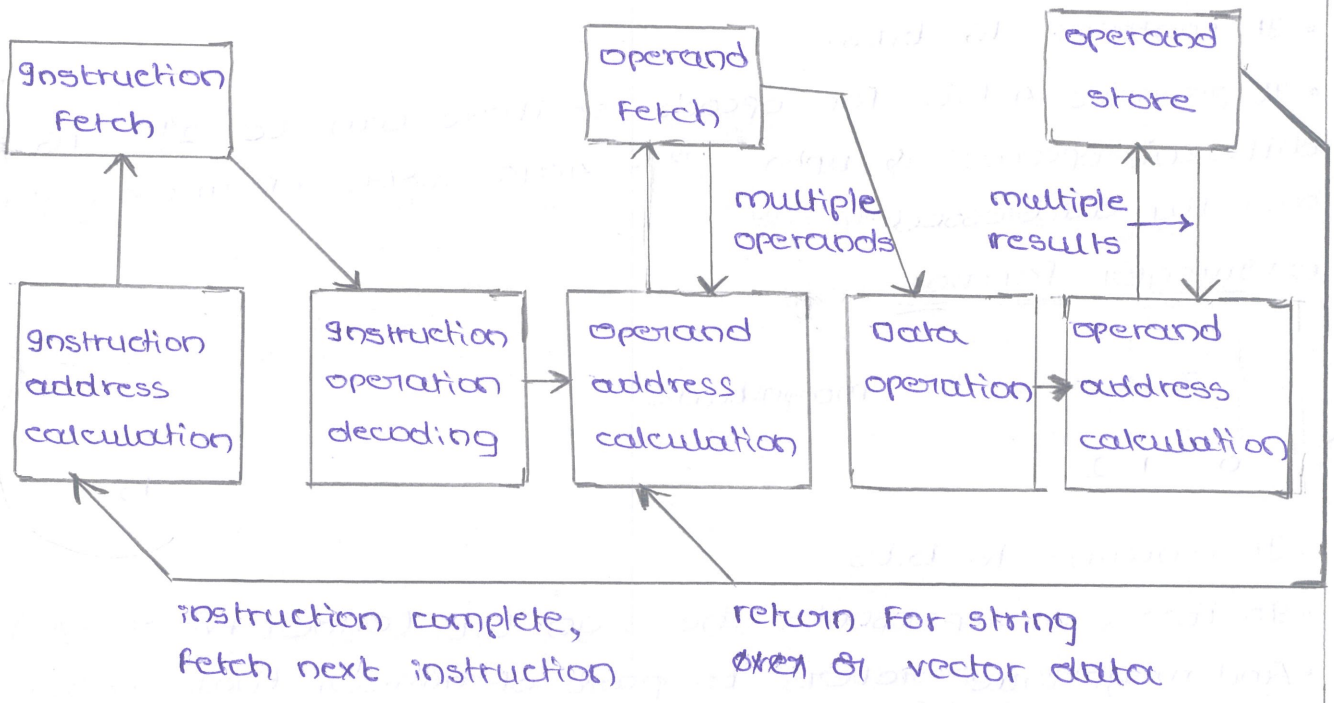
(10) Result storage : If necessary, the result of operation is stored back to memory

(11) PC updates : The PC is incremented to point to the next instruction to be executed.

Instruction cycle

* The execution cycle for a particular instruction may involve more than one reference to memory. Also, instead of memory references, an instruction may specify an I/O operation.

* For any given instruction cycle, some states may be null & others may be visited more than once.



*The states can be described as follows:

(1) Instruction address calculation: Determine the address of next instruction to be executed. Usually, this involves adding a fixed number to address of previous instruction.

(2) Instruction fetch: Read instruction from its memory location into processor.

(3) Instruction operation decoding: Analyze instruction to determine type of operation to be performed & operand to be used

(4) Operand address calculation: If operation involves reference to an operand in memory or available (via I/O), then determine address of operand.

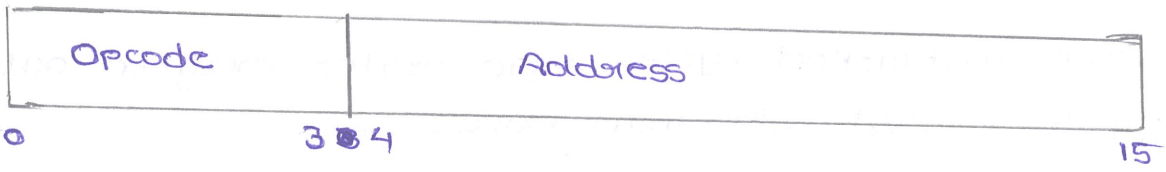
(5) Operand Fetch: Fetch operand from memory or read it from I/O.

(6) Data operation: Perform the operation indicated in the instruction.

(7) Operand store: Write the result into memory or out to I/O.

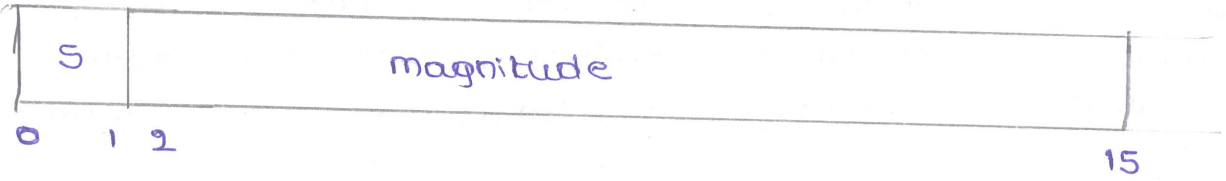
Characteristics of Hypothetical Machine

(1) Instruction format:



- It contains 16 bits.
- It provides 4 bits for opcode, so there can be $2^4 = 16$ different opcodes & upto $2^{12} = 4096$ words of memory can be directly addressed.

(2) Integer format:



- It contains 16 bits
- In this, S represents the sign bit (either +ve or -ve)
- And magnitude refers to part of number that stores the

the absolute value when using a signed representation (sign bit).

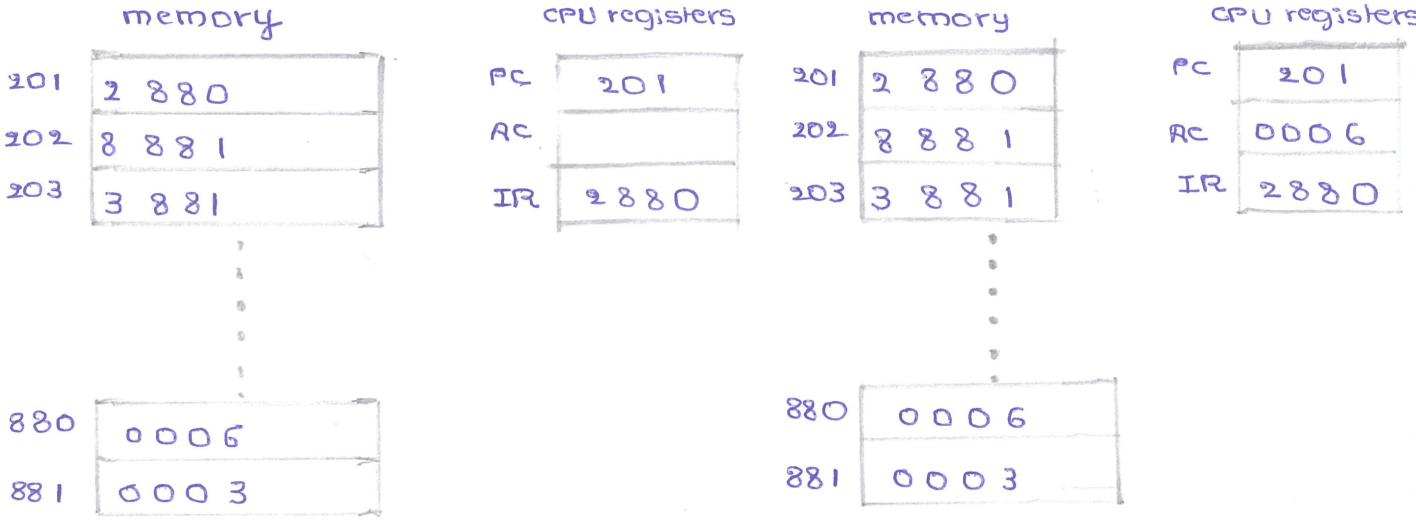
(3) Internal CPU registers:

- Program counter (PC) = Address of instruction
- Instruction register (IR) = Instruction being executed
- Accumulator (AC) = Temporary storage

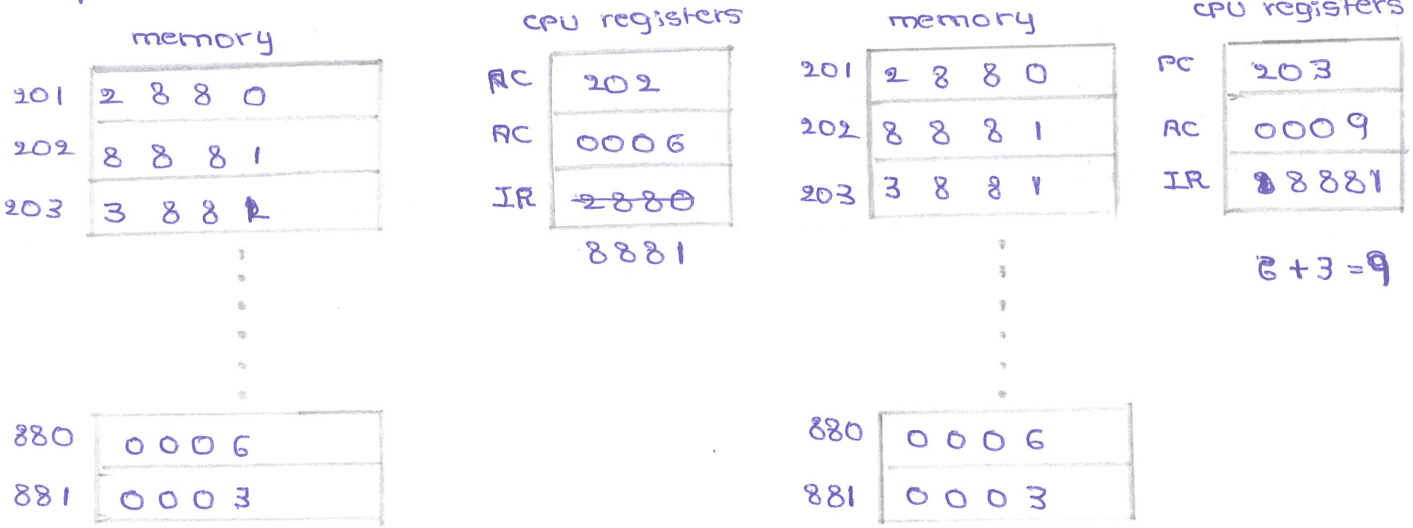
(4) Partial list of opcodes:

- 0010 = load AC from memory (2)
- 0011 = store AC to memory (3)
- 1000 = add to AC from memory (8)

Example of Program execution:



Step - 1



Step - 2

Step - 3

Step - 4

memory

201	2 8 8 0
202	8 8 8 1
203	3 8 8 1

CPU registers

PC	203
AC	0009
IR	~~8 8 8 1~~ 3 8 8 1

880	0 0 0 6
881	0 0 0 3

Step - 5

memory

201	2 8 8 0
202	8 8 8 1
203	3 8 8 1

CPU registers

PC	204
AC	0009
IR	3 8 8 1

880	0 0 0 6
881	0 0 0 9

Step - 6

Signal :

* A signal is a function that represents variation of physical quantity with respect to any parameter
* Generally, the parameter is time or diameter
* In electrical & electronics, the physical quantity is current & voltage.
* The signals are 3 types, they are :

(1) Analog signal :

- The signals that can take any value between two limits
- It is continuous in nature.
- All naturally occuring physical phenomena are analog signals

(2) Discrete time signal :

- The signal that is defined for discrete intervals of time
- It is a subset of analog signals

(3) Digital signal :

- In this, both time and magnitude are discretized.
- Discretize the time axis into equal intervals.
- Discretize the magnitude axis into a fixed number of levels.
- select the lower level to minimize the error.
- Error will reduced if no.of levels are increased.
- The need for digital signals is minimizing the noise (i.e; unwanted signal)

Boolean Algebra :

* Mathematical discipline used to design & analyze the behaviour of digital circuitry in digital computers & other digital systems
* Named after George Boole is an English mathematician who proposed basic principles of this algebra in 1854.
* Is a convenient tool :

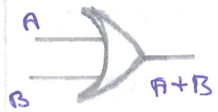
- Analysis : It is an economical way of describing the functions of digital circuitry.

• Design : given a desired function, boolean algebra can be applied to develop a simplified implementation of that function.

Boolean variable :

* It is a digital signal.
* It is denoted by A - Z (or) a - z and it contains only two values (i.e, (0,1))
* If A = 0, then A' (or) $\bar{A}$ = 1 (it is a negative logic)
  If A = 1, then A' (or) $\bar{A}$ = 0 (it is a positive logic)

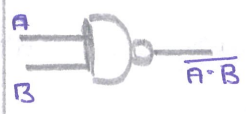
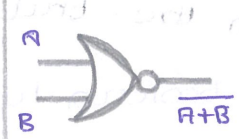
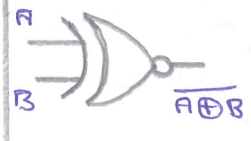
Basic operations that can be done in boolean algebra :

S.No	Operation	graphic symbol	Operator	Truth table	Boolean functions
1.	AND		·	A B A·B 0 0 0 0 1 0 1 0 0 1 1 1	F(A,B) = A·B
2.	OR		+	A B A+B 0 0 0 0 1 1 1 0 1 1 1 1	F(A,B) = A+B
3.	NOT		A' & $\bar{A}$	A A' & $\bar{A}$ 0 1 1 0	F(A,B) = A' (or) $\bar{A}$

Other operations that can be done in boolean algebra :

* The other operations are

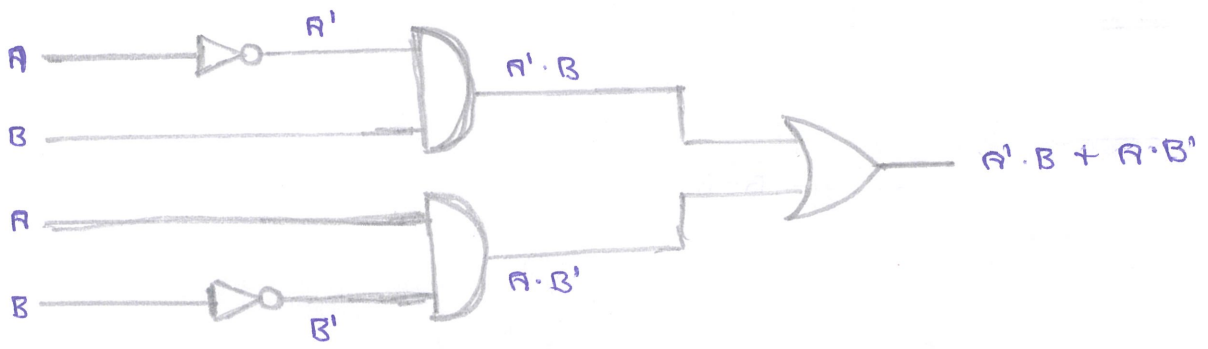
(1) NAND (2) NOR (3) XOR (4) XNOR

S.No	Operation	Graphic symbol	Operator	Truth table	Boolean Function
1.	NAND		–	A B $\overline{A \cdot B}$ 0 0 1 0 1 1 1 0 1 1 1 0	$F(A,B) = \overline{A \cdot B}$
2.	NOR		–	A B $\overline{A+B}$ 0 0 1 0 1 0 1 0 0 1 1 0	$F(A,B) = \overline{A+B}$
3.	XOR		$\oplus$	A B $A \oplus B$ 0 0 0 0 1 1 1 0 1 1 1 0	$F(A,B) = A \oplus B$ $= A'B + AB'$
4.	XNOR		$\odot$	A B $A \odot B$ 0 0 1 0 1 0 1 0 0 1 1 1	$F(A,B) = A \odot B$ $= \overline{A \oplus B}$ $= A'B' + AB$

* XOR operation is also called odd number of one's operator
* SOP (sum of products) : If two or more than two product terms are ORed
* POS (Product of sums) : If two or more than two sum terms are ANDed

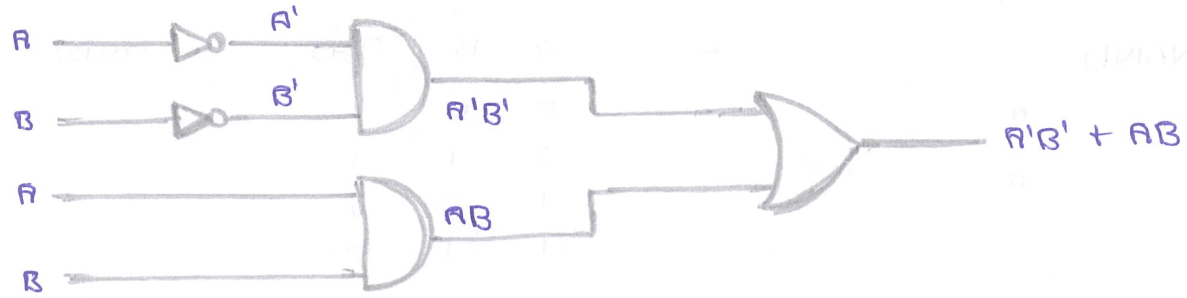
Q : Draw XOR using AND, OR and NOT.

Sol:



Q: Draw XNOR using AND, OR and NOT

Sol:



Q: How to get a boolean function from the truth table?

Sol: There are some steps for getting a boolean function from the truth table.

Step-1 : Draw the truth table

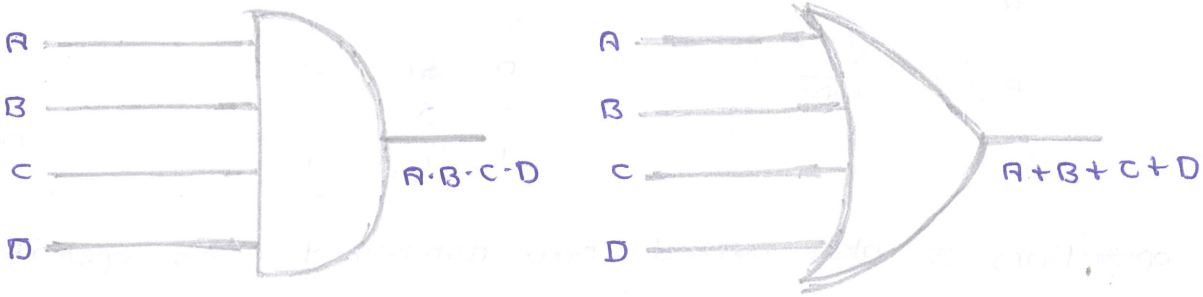
Step-2 : Identify the combinations wherever it is 1 (output)

Step-3 : Use positive logic SOP of each combinations

Step-4 : OR those SOP terms

Q: Is it possible to have gates with multiple inputs?

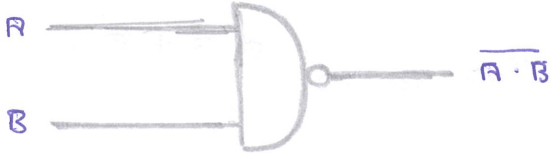
Sol:



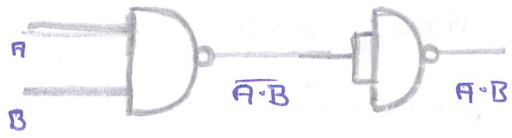
Q: Prove that NAND & NOR gates are universal gates.

Sol: Universal gate : A gate can be ~~used~~ terminated as universal gate if it can be used to implement all these basic gates (i.e; AND, OR, NOT)

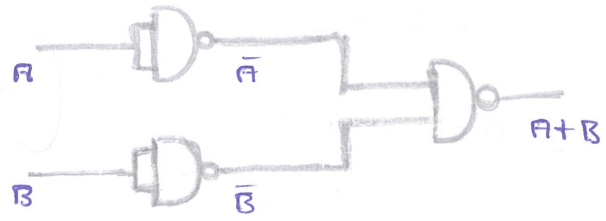
NAND gate :



- **AND gate** : $\overline{(A \cdot B)} = A \cdot B \quad (\because (A')' = A)$



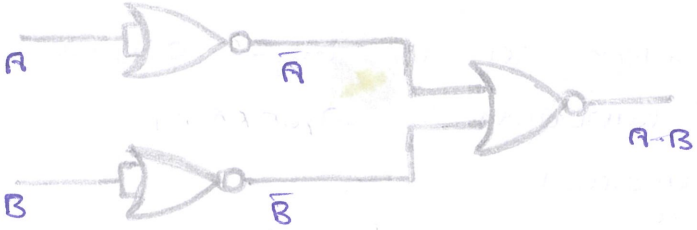
- **OR gate** : $\overline{(\bar{A} \cdot \bar{B})} = (\bar{\bar{A}}) + (\bar{\bar{B}}) = A + B$



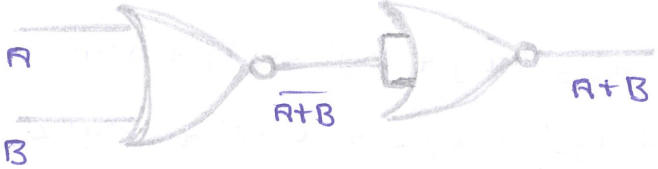
- **NOT gate** : complement of NAND gate of A is $\bar{A}$

NOR gate :

- **AND gate** : $\overline{(\bar{A} + \bar{B})} = (\bar{\bar{A}}) \cdot (\bar{\bar{B}}) = A \cdot B$



- **OR gate** : $\overline{(\overline{A+B})} = A+B \quad (\because (A')' = A)$



Basic identities of boolean algebra :

commutative law	$A \cdot B = B \cdot A$	$A + B = B + A$
distributive law	$A \cdot (B+C) = (A \cdot B) + (A \cdot C)$	$A + (B \cdot C) = (A+B) \cdot (A+C)$
associative law	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B+C) = (A+B) + C$
demorgan's theorem	$\overline{A \cdot B} = \bar{A} + \bar{B}$	$\overline{A+B} = \bar{A} \cdot \bar{B}$
identity elements	$1 \cdot A = A$	$0 + A = A$
inverse elements	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$
other elements	$0 \cdot A = 0$	$1 + A = 1$
	$A \cdot A = A$	$A + A = A$
absorbtion law	$A + AB = A \cdot 1 + A \cdot B = A(1+B) = A$	

Q: What is a canonical form ?

Sol: For given boolean function, every term of it contains all the literals of the function in its ~~digit~~ original or complement form. Then the function is said to be canonical form (sum of minterms)

Ex: $F(x, y, z) = xy + z'$

$= x \cdot y \cdot 1 + z' \cdot 1 \cdot 1$

$= x \cdot y \cdot (z + z') + z' \cdot (x + x') \cdot (y + y')$

$= xyz + xyz' + z'(xy + xy' + x'y + x'y')$

$= xyz + \underbrace{xyz' + xyz'} + xy'z' + x'yz' + x'y'z'$

$= xyz + xyz' + xy'z' + x'yz' + x'y'z'$

$F(x,y,z) = xyz + xyz' + xy'z' + x'yz' + x'y'z'$

$111 \quad 110 \quad 100 \quad 010 \quad 000$

$= \sum_m (0,2,4,6,7)$

Min terms (Product terms) and Max terms (sum terms):

A	B	C	Min terms (sop)	Max terms (POS)
0	0	0	$A'B'C'$ (m_0)	$A+B+C$ (M_0)
0	0	1	$A'B'C$ (m_1)	$A+B+C'$ (M_1)
0	1	0	$A'BC'$ (m_2)	$A+B'+C$ (M_2)
0	1	1	$A'BC$ (m_3)	$A+B'+C'$ (M_3)
1	0	0	$AB'C'$ (m_4)	$A'+B+C$ (M_4)
1	0	1	$AB'C$ (m_5)	$A'+B+C'$ (M_5)
1	1	0	ABC' (m_6)	$A'+B'+C$ (M_6)
1	1	1	ABC (m_7)	$A'+B'+C'$ (M_7)

Ex: (1) $F(A,B,C) = m_1 + m_2 + m_4$

$= A'B'C + A'BC' + AB'C'$

$= \sum_m (1,2,4)$

(2) converting min terms into max terms using demorgan's law

$(A'B'C')' = (A')' + (B')' + (C')' = A+B+C$

(3) canonical forms also can be represented as POS

$F(A,B,C) = M_1 \cdot M_2 \cdot M_4$

$= (A+B+C') \cdot (A+B'+C) \cdot (A'+B+C)$

$= \prod_M (1,2,4)$

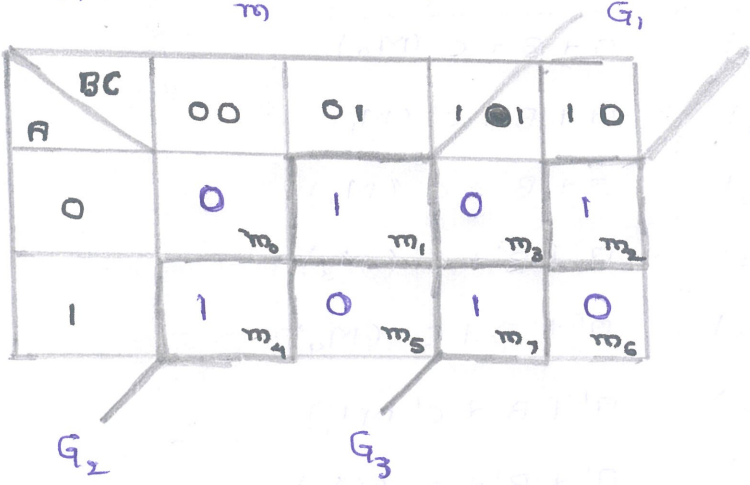
* Min terms are also called AND-OR gates. Min terms are taking positive logic.
* Max terms are also called OR-AND gates. Max terms are taking negative logic.

Karnaugh maps :

* They are graphical way of minimizing a boolean function. They are of different types of catagorized according to the number of literals in boolean function.

case (i) :- Group of isolated cells

$F(A,B,C) = \sum_m (1,2,4,7)$



$F = G_1 + G_2 + G_3 + G_4$

$F = A'B'C + AB'C' + ABC + A'BC'$

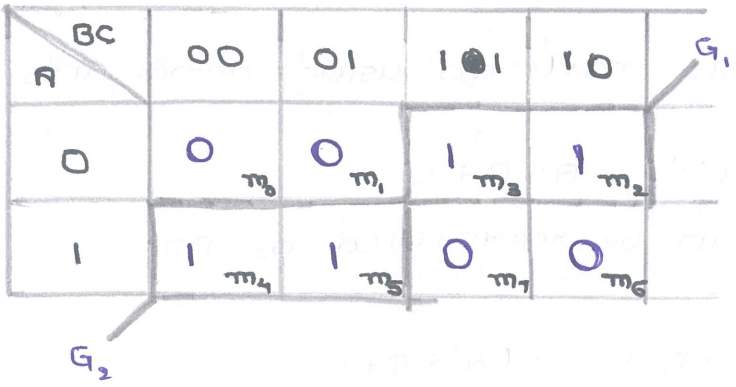
$F = A \oplus B \oplus C$

** The grouping cannot be done by diagonally

* This case was named as grouped isolated cells

Case (ii) :- Group of 2 - 1's

$F(A,B,C) = \sum_m (2,3,4,5)$



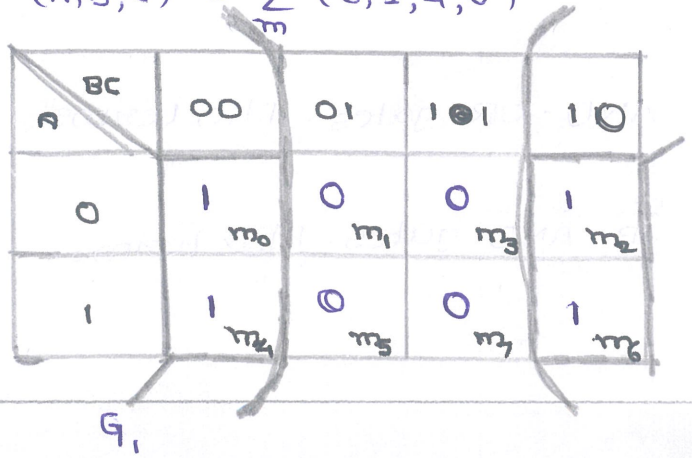
$F = G_1 + G_2$

$F = A'BC + A'BC' + AB'C' + AB'C$

$F = A'B + AB'$

$F = A \oplus B$

$F(A,B,C) = \sum_m (0,2,4,6)$



$F = G_1 + G_2$

$F = A'B'C' + AB'C' + A'BC' + ABC'$

$F = B'C' + BC'$

$F = C'$

case (iii) :- Group of 4 - 1's

$F = \sum_m (0, 1, 4, 5)$

A \ BC	00	01	11	10
0	1 m_0	1 m_1	0 m_3	0 m_2
1	1 m_4	1 m_5	0 m_7	0 m_6

G_1

$F = G_1$

$F = A'B'C' + A'B'C + AB'C' + AB'C$

$F = A'B'(C'+C) + AB'(C'+C)$

$F = A'B' + AB' = B'$

case (iv) :- Group of 8 - 1's

$F = \sum_m (0, 1, 2, 3, 4, 5, 6, 7)$

A \ BC	00	01	11	10
0	1 m_0	1 m_1	1 m_3	1 m_2
1	1 m_4	1 m_5	1 m_7	1 m_6

G_1

$F = G_1$

$F = \underline{A'B'C' + AB'C'} + \underline{A'B'C + AB'C} + \underline{A'BC + ABC} + \underline{A'BC' + ABC'}$

$F = \underline{B'C' + B'C'} + \underline{B'C + B'C} + \underline{BC + BC} + \underline{BC' + BC'}$

$F = B'C' + B'C + BC + BC'$

$F = B'(C'+C) + B(C+C')$

$F = B' + B$

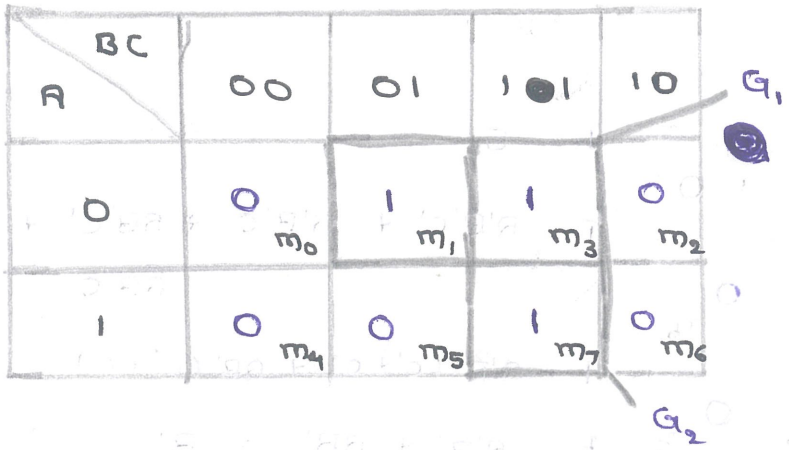
$F = 1$

Overlapping :

(1) $F = \sum_m (1, 3, 7)$

$F = G_1 + G_2$

$F = A'B'C + A'BC + ABC$



$F = A'B'C + BC(A'+A)$

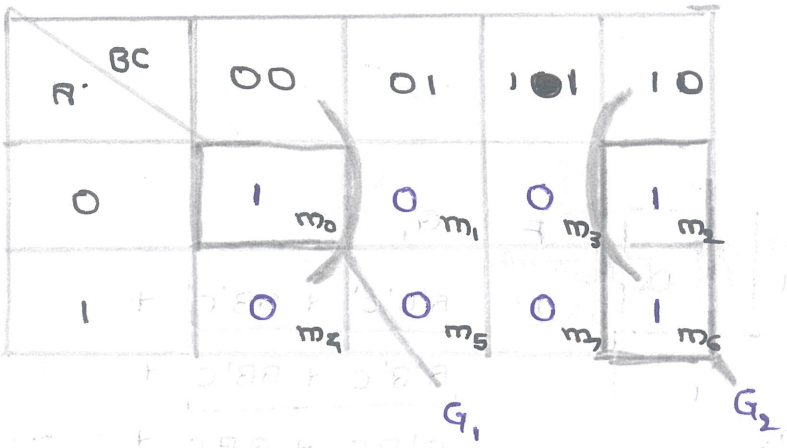
$F = A'B'C + BC$

$F = C(A'B' + B)$

$F = C((A'+B)\cdot(B'+B))$

$F = A'C + BC$ (using distributive law)

(2) $F = \sum_m (0, 2, 6)$



$F = G_1 + G_2$

$F = A'C' + BC'$

$F = C'(A' + B)$

(3) $F = \sum_m (0, 2, 3, 6)$

$F = G_1 + G_2 + G_3$

$F = A'C' + BC' + A'B$

* The boolean Function "$A'C' + BC' + A'B$" is a <u>concensus theorem</u>

* If a '1' is grouped with all neighbouring groups then it forms a redundant group. In such cases, isolated grouping is required.